QML Lab • research project 01

Quantum-Kernel Intrusion Detection

Can a quantum kernel separate normal from malicious network traffic better than a tuned classical kernel?

Living document • last updated October 25, 2026

Quadrant	CQ (classical data, quantum processor)	**Track**	Kernels
Difficulty	Medium	**Tools**	Qiskit, scikit-learn
Builds on	Modules 4 and 6 of the QML Lab	**Workspace**	`qml_projects/quantum-kernel-intrusion-detection/`

Summary. This project applies a quantum-kernel support vector machine to network intrusion detection, a security task with real, imbalanced data. Traffic records are reduced to a few features with PCA, encoded into qubits with a ZZ feature map, and classified with an SVM that uses the state overlap as its kernel. The comparison that matters is against an RBF kernel and a small neural network with the same features. Kernel bandwidth, the scaling of features before encoding, is treated as the main control knob, because it decides whether the quantum kernel generalises or simply memorises.

1 Background and key concepts

1.1 Intrusion detection as classification

A network intrusion detection system labels connections or flows as normal or as one of several attack types. Public benchmarks make results comparable. NSL-KDD (Tavallaee et al., 2009) removes the duplicate records that inflated results on the older KDD Cup 99 data and has 41 features per connection and four attack families (DoS, Probe, R2L, U2R). CICIoT2023 (Neto et al., 2023) is a newer data set recorded on a real IoT network, with many attack types and flow-level features. Both are strongly imbalanced: some attack classes are rare, so accuracy alone is misleading.

1.2 Support vector machines and kernels

An SVM (Cortes and Vapnik, 1995) learns a maximum-margin separator. In its dual form it only needs the kernel $k(x, x')$ between pairs of training points:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \, k(x_i, x_j) \quad \text{s.t.} \quad 0 \le \alpha_i \le C, \;\; \sum_i \alpha_i y_i = 0, \tag{1}$$

and predicts $f(x) = \operatorname{sign}\big(\sum_i \alpha_i y_i k(x_i, x) + b\big)$. Any positive semi-definite similarity can be plugged in, including one computed by a quantum circuit.

1.3 The quantum kernel

The ZZ feature map of Havlíček et al. (2019) encodes $x \in \mathbb{R}^d$ on d qubits with Hadamards and phases that depend on single features and on pairwise products. The kernel is the fidelity of

two encoded states,

$$\kappa(x, x') = |\langle\psi(x)|\psi(x')\rangle|^2 = |\langle 0|U_\Phi(x')^\dagger U_\Phi(x)|0\rangle|^2, \tag{2}$$

estimated on hardware as the probability of measuring all zeros after running $U_\Phi(x')^\dagger U_\Phi(x)$. Building the Gram matrix needs $O(N^2)$ circuit evaluations for N training points, which limits training sets to a few thousand points.

1.4 Bandwidth and concentration

Before encoding, features are scaled as $x \mapsto \lambda x$. For large λ the encoded states are almost orthogonal, the Gram matrix approaches the identity and the model memorises the training set; this is exponential concentration (Thanasilp et al., 2024). Shaydulin and Wild (2022) showed that reducing the bandwidth λ restores generalisation, in the same way as the width of an RBF kernel. Huang et al. (2021) give a test for whether a quantum kernel can beat classical ones at all on given data: the *geometric difference*

$$g(K^C \| K^Q) = \sqrt{\left\| \sqrt{K^Q}\,(K^C)^{-1}\sqrt{K^Q} \right\|_\infty}, \tag{3}$$

which must be large for a quantum advantage to be possible.

2 Research questions

The project is open-ended: these questions are the starting point, not the finish line. Add new ones to the README and the journal as they come up.

RQ1 At the best bandwidth, does the quantum-kernel SVM beat a tuned RBF-SVM and a small MLP on binary detection with the same 4–8 features?

RQ2 Does the quantum kernel help on rare attack classes (per-class recall), or only on the easy majority classes?

RQ3 How do accuracy, kernel concentration and the geometric difference change with bandwidth λ and qubit number d?

3 Method

3.1 Data

NSL-KDD first (KDDTrain+ for training, KDDTest+ for testing), then CICIoT2023. Categorical features are one-hot encoded, all features standardised, and PCA reduces them to $d \in \{4, 6, 8\}$ components, scaled to $[0, \pi]$ and then multiplied by λ. Because the Gram matrix costs $O(N^2)$ circuit runs, training uses stratified subsamples of $N = 250$ to 2000 connections; the test set keeps its original class balance.

3.2 Models

Fidelity kernel with the ZZ feature map (`reps` 1–2), simulated exactly with state vectors; the precomputed Gram matrix goes into `sklearn.svm.SVC` (Pedregosa et al., 2011) with C chosen by cross-validation. Multi-class detection uses one-vs-rest.

3.3 Baselines

RBF-SVM with (C, γ) tuned on the same features; logistic regression; an MLP with a similar number of parameters; and, as a sanity check, the quantum kernel with $\lambda \to 0$, which approaches a constant kernel.

3.4 Metrics

- Accuracy, macro-F1, attack recall and false-positive rate; ROC-AUC for the binary task.
- Per-class recall for the four NSL-KDD attack families.
- Kernel diagnostics: off-diagonal variance, kernel–target alignment and geometric difference g.
- Cost: number of circuit evaluations and wall-clock time.

4 Planned experiments

Each experiment becomes one `[[experiments]]` entry in the workspace, so its runs, parameters and metrics are stored and compared in the portal.

ID	Experiment	What varies	Main metrics
E1	Binary detection, $d = 4$	bandwidth $\lambda \in \{0.05, \dots, 1\}$	macro-F1, off-diagonal variance
E2	Qubit scaling	$d \in \{4, 6, 8\}$ at the best λ	macro-F1, g, run time
E3	Attack families	one-vs-rest over DoS, Probe, R2L, U2R	per-class recall
E4	Learning curves	$N \in \{250, 500, 1000, 2000\}$	test macro-F1 vs RBF
E5	Shot noise	kernel entries from M shots	accuracy loss vs exact kernel

Table 1: Experiment plan. Results go into the journal and the results log (section 9).

5 Running it in the QML Lab

The workspace lives in `qml_projects/quantum-kernel-intrusion-detection/`. Scripts import three helpers from `qml_lab/plots.py`:

- `params()` reads the parameters from the portal's run form (defaults when run by hand);
- `save()` stores figures with the run;
- `log_metrics()` records numbers that appear in the run table.

```
qml_projects/quantum-kernel-intrusion-detection/
  project.toml          metadata + experiments
  README.md             goal, open questions, ideas
  experiments/          runnable scripts
  docs/main.tex         this document
```

First experiment: This experiment is *planned*: write the script, add the block below to `project.toml`, then run `python manage.py sync_projects`.

```
[[experiments]]
slug = "nsl-kdd-binary"
title = "Binary NSL-KDD detection with a ZZ quantum kernel"
script = "experiments/e01_nsl_kdd_binary.py"
figure = "kernel_vs_rbf.png"

[experiments.params]
n_features = 4
bandwidth = 0.5
reps = 2
n_train = 1000
seed = 0
```

```
# from the portal folder: pick up changes to project.toml
python manage.py sync_projects
cd qml_projects/quantum-kernel-intrusion-detection
# run by hand with the default parameters
PYTHONPATH=../../qml_lab python experiments/e01_nsl_kdd_binary.py
```

6 Quality bar

- A classical baseline with a similar number of trainable parameters, tuned with the same care (Bowles et al., 2024).
- At least five random seeds per configuration, reported as mean $\pm$ standard deviation.
- How results change with the number of qubits.
- At least one run under a realistic noise model.
- Report per-class recall and false-positive rate, not accuracy alone; the data is imbalanced.

7 Open directions

- Move from NSL-KDD to CICIoT2023 and compare how conclusions change on recent IoT traffic.
- One-class SVM with the quantum kernel for anomaly detection when attack labels are missing.
- Compare inference cost and accuracy with a TinyML-sized classical detector running on an edge device.
- Projected quantum kernels (Project 03) as a less concentrated alternative.

8 Risks and limitations

- $O(N^2)$ kernel evaluations limit training-set size; subsampling may favour simpler models.
- PCA to 4–8 components can discard the signal of rare attacks; check what the baselines lose too.

- NSL-KDD has known artefacts and is old; treat it as a benchmark, not as evidence of real-world performance.
- Large bandwidth or many qubits leads to concentrated, memorising kernels.

9 Results log

Summarise findings here as they become stable; day-to-day notes belong in the portal journal.

Date	Run	Finding	Next step

References

Bowles, J., Ahmed, S. and Schuld, M. (2024) 'Better than classical? The subtle art of benchmarking quantum machine learning models', Preprint, arXiv:2403.07059. Available at: https://arxiv.org/abs/2403.07059.

Cortes, C. and Vapnik, V. (1995) 'Support-vector networks', *Machine Learning*, 20(3), pp. 273–297. Available at: https://doi.org/10.1007/BF00994018.

Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M. and Gambetta, J. M. (2019) 'Supervised learning with quantum-enhanced feature spaces', *Nature*, 567, pp. 209–212. Available at: https://doi.org/10.1038/s41586-019-0980-2.

Huang, H.-Y., Broughton, M., Mohseni, M., Babbush, R., Boixo, S., Neven, H. and McClean, J. R. (2021) 'Power of data in quantum machine learning', *Nature Communications*, 12, 2631. Available at: https://doi.org/10.1038/s41467-021-22539-9.

Neto, E. C. P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R. and Ghorbani, A. A. (2023) 'CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment', *Sensors*, 23(13), 5941. Available at: https://doi.org/10.3390/s23135941.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Shaydulin, R. and Wild, S. M. (2022) 'Importance of kernel bandwidth in quantum machine learning', *Physical Review A*, 106(4), 042407. Available at: https://doi.org/10.1103/PhysRevA.106.042407.

Tavallaee, M., Bagheri, E., Lu, W. and Ghorbani, A. A. (2009) 'A detailed analysis of the KDD CUP 99 data set', in *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, IEEE, pp. 1–6. Available at: https://doi.org/10.1109/CISDA.2009.5356528.

Thanasilp, S., Wang, S., Cerezo, M. and Holmes, Z. (2024) 'Exponential concentration in quantum kernel methods', *Nature Communications*, 15, 5200. Available at: https://doi.org/10.1038/s41467-024-49287-w.